Distributed Checkpoints & Elastic Training

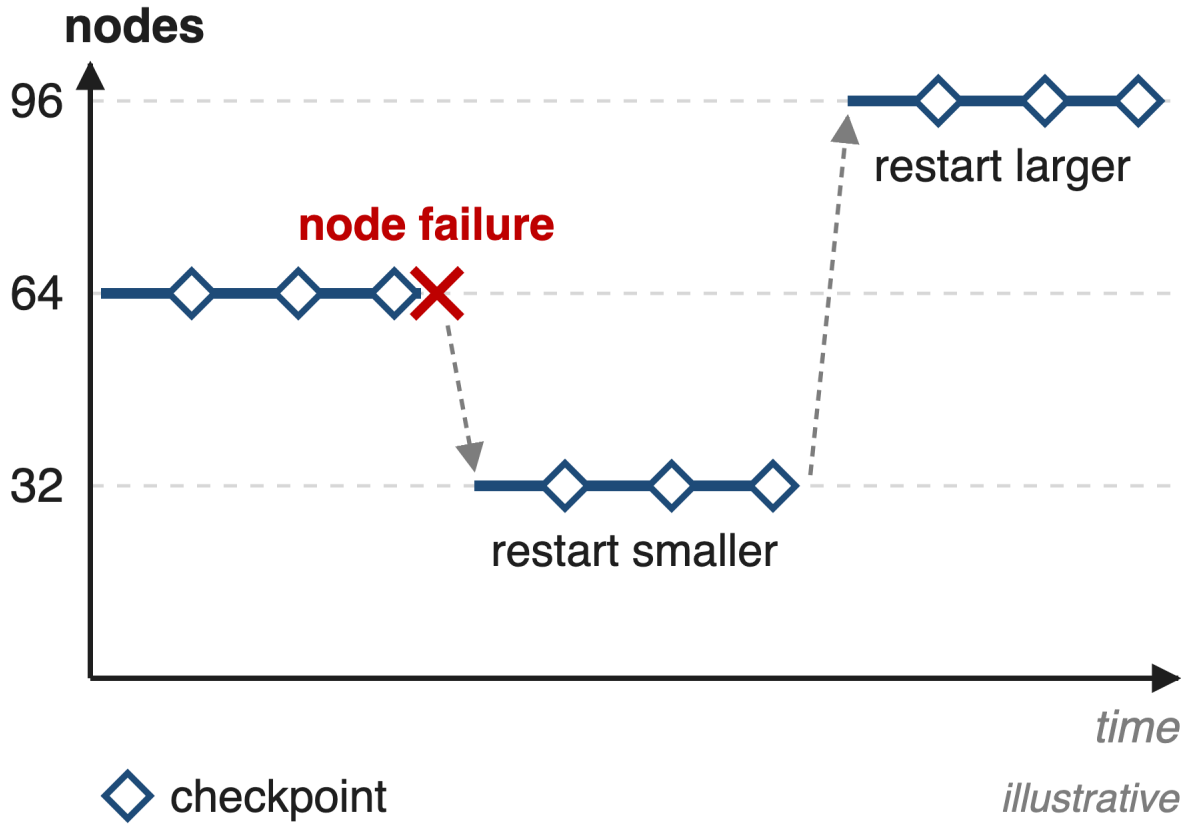
Save shards once; resume under any layout or GPU count

VeOmni (ByteDance Seed) · Kimi K2 (Moonshot AI) · PyTorch DCP

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Long runs restart on whatever GPUs remain

- Kimi K2: **1.04T** MoE, **15.5T** tokens on H800
- Built to run on any multiple of **32** nodes
- Every restart reloads the last checkpoint



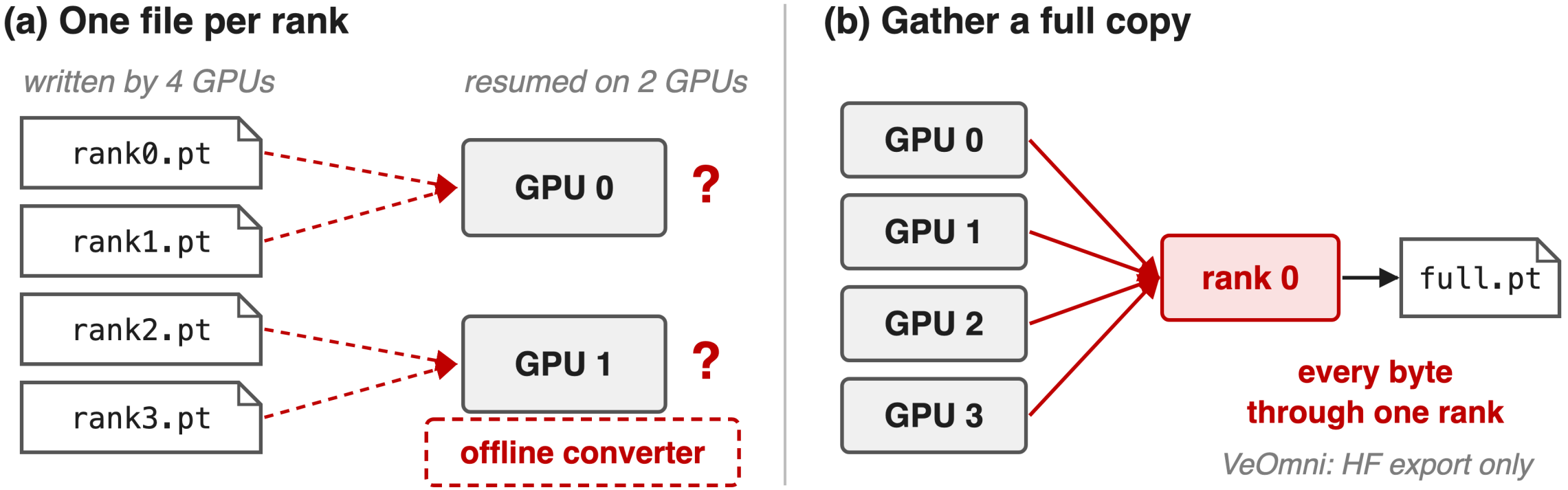
Even async checkpoint saves stall every GPU

18.5 s

training stall per checkpoint: Llama 3 405B, 432 H200 GPUs

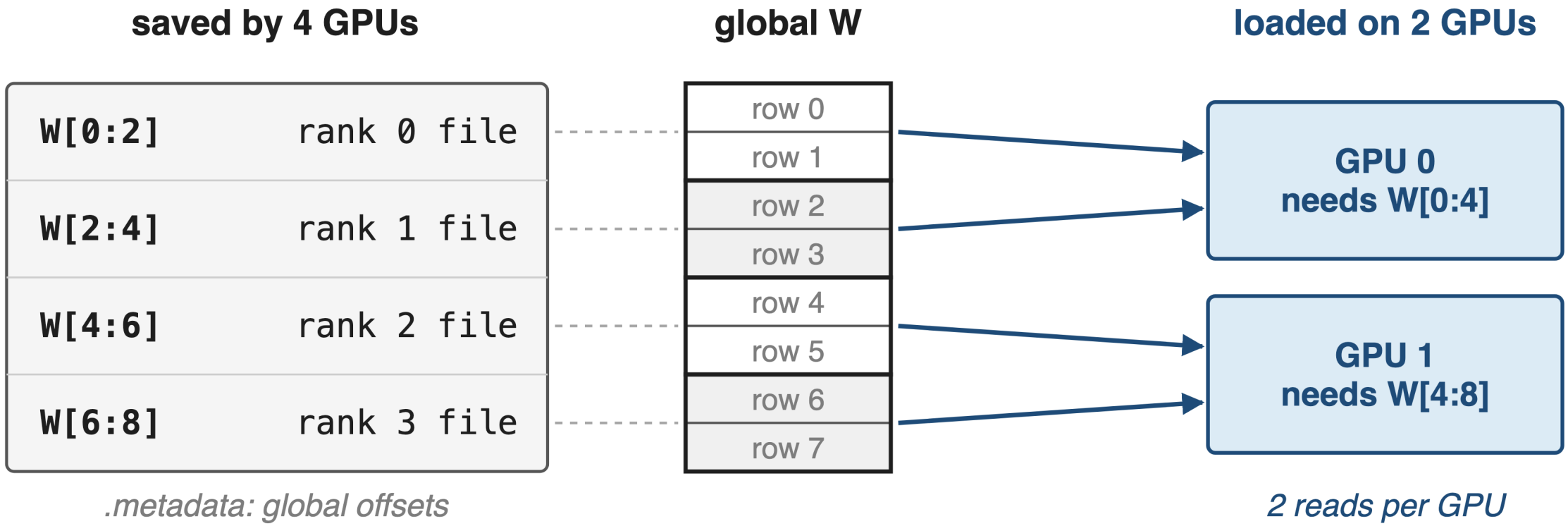
PyTorch blog "Distributed Checkpoint: Efficient checkpointing in large-scale jobs" (Sep 2025), results table

Rank-file checkpoints fit only one layout



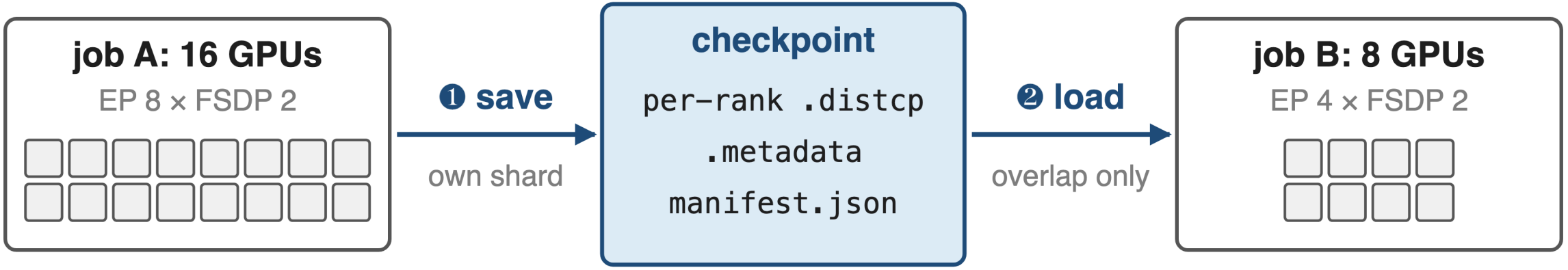
A per-rank file records who held the bytes, not which slice they are

Insight: a shard is a slice of a global tensor



Record global offsets once; any new layout is just a different set of reads

Save once, then load into any mesh



❸ Elastic layout (Kimi K2)

fixed 256-GPU model-parallel unit + ZeRO-1 DP that grows or shrinks

❶❷ PyTorch DCP as used by VeOmni (parent deck 28); ❸ Kimi K2's layout choice. Mesh sizes are the running example

❶ Every rank writes only its own shard

global_step_N/	holds	written by
model/ckpt/	weights	**all ranks (DCP)**
model/optimizer/	optimizer state	**all ranks (DCP)**
model/lr_scheduler.pt	LR schedule	rank 0
loader/rank_R.pt	data cursor	rank R
extra_state/rank_R.pt	step and RNG	rank R
checkpoint_manifest.json	done marker	rank 0 (last)

A step counts only when the manifest and every DCP .metadata exist

❷ The loader reads only overlapping chunks

runtime shard (EP dim dropped)
`[16, 1024, 768]`

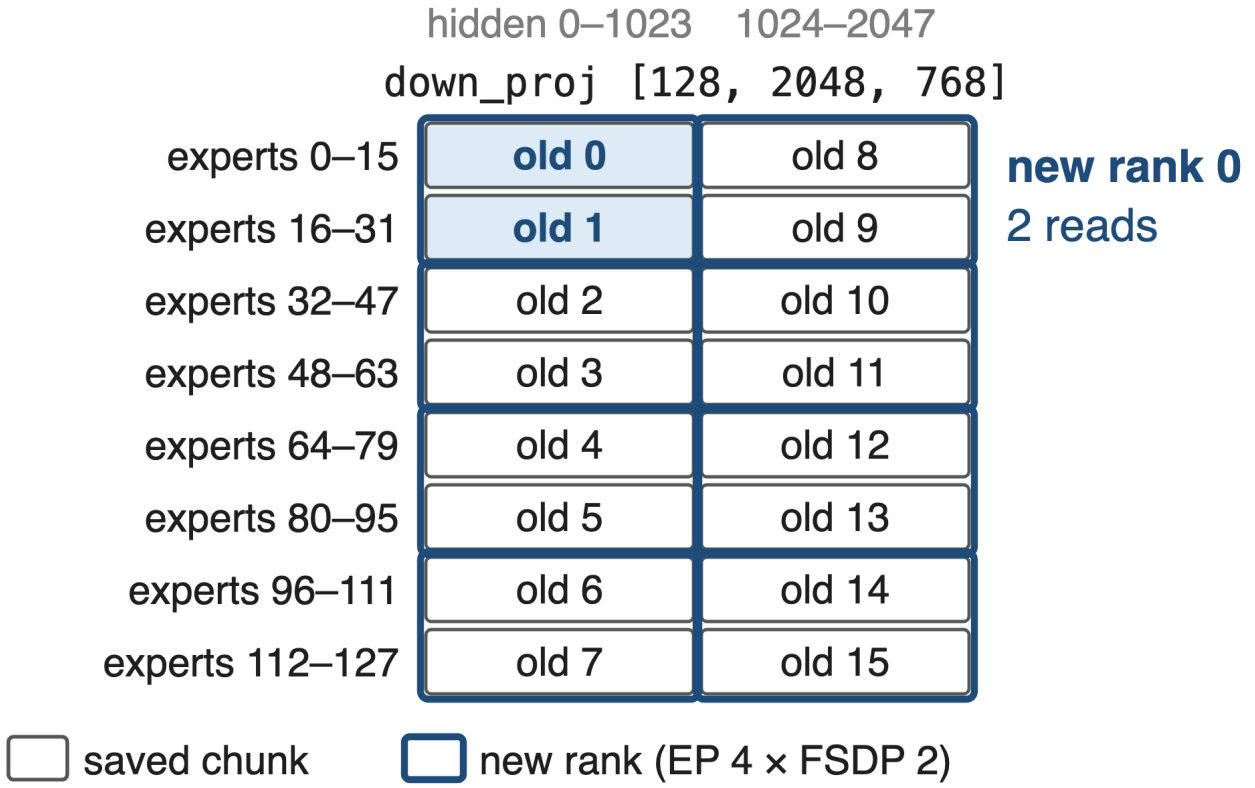
↓

save: restore EP dim
`[Shard(1), Shard(0)]`

↓

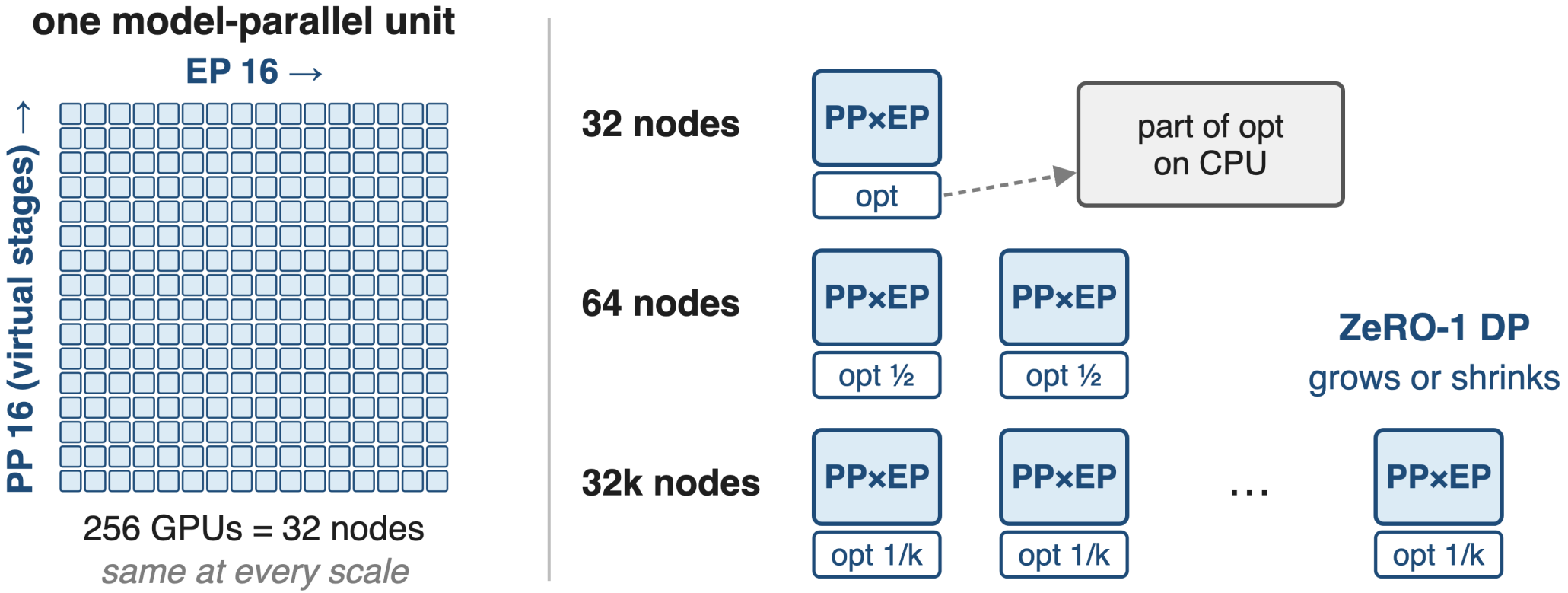
DCP records global tensor
`[128, 2048, 768]`

load: drop EP dim again



Old layout from VeOmni's EP+FSDP2 docs (Qwen3-30B-A3B); new layout illustrative, same rule EP × FSDP = world size

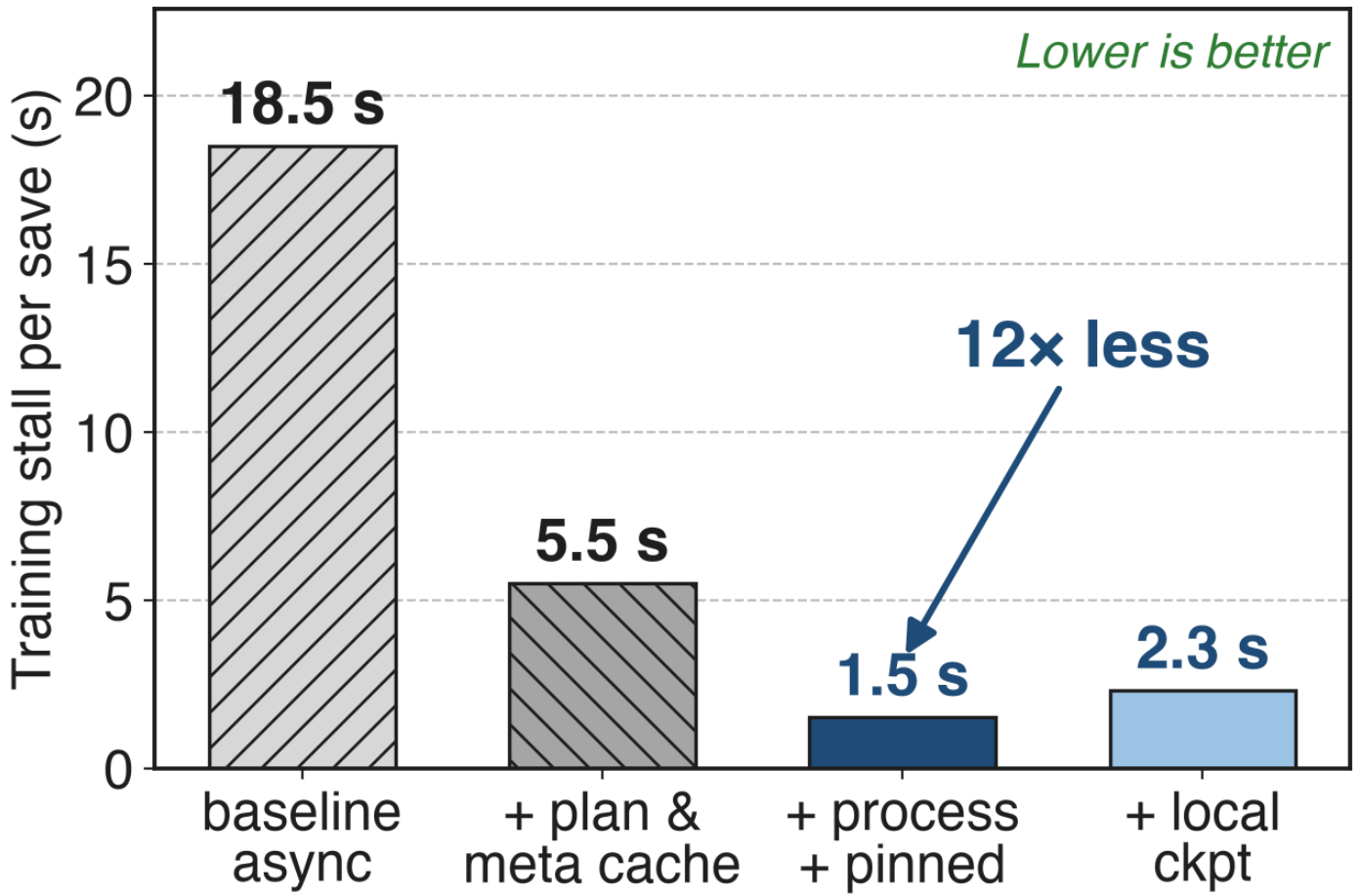
❸ K2 fixes model parallelism, flexes only DP



Kimi K2 report §2.4.2: ~6 TB of BF16 weights + FP32 grad buffers per 256-GPU unit; ~30 GB of state per GPU

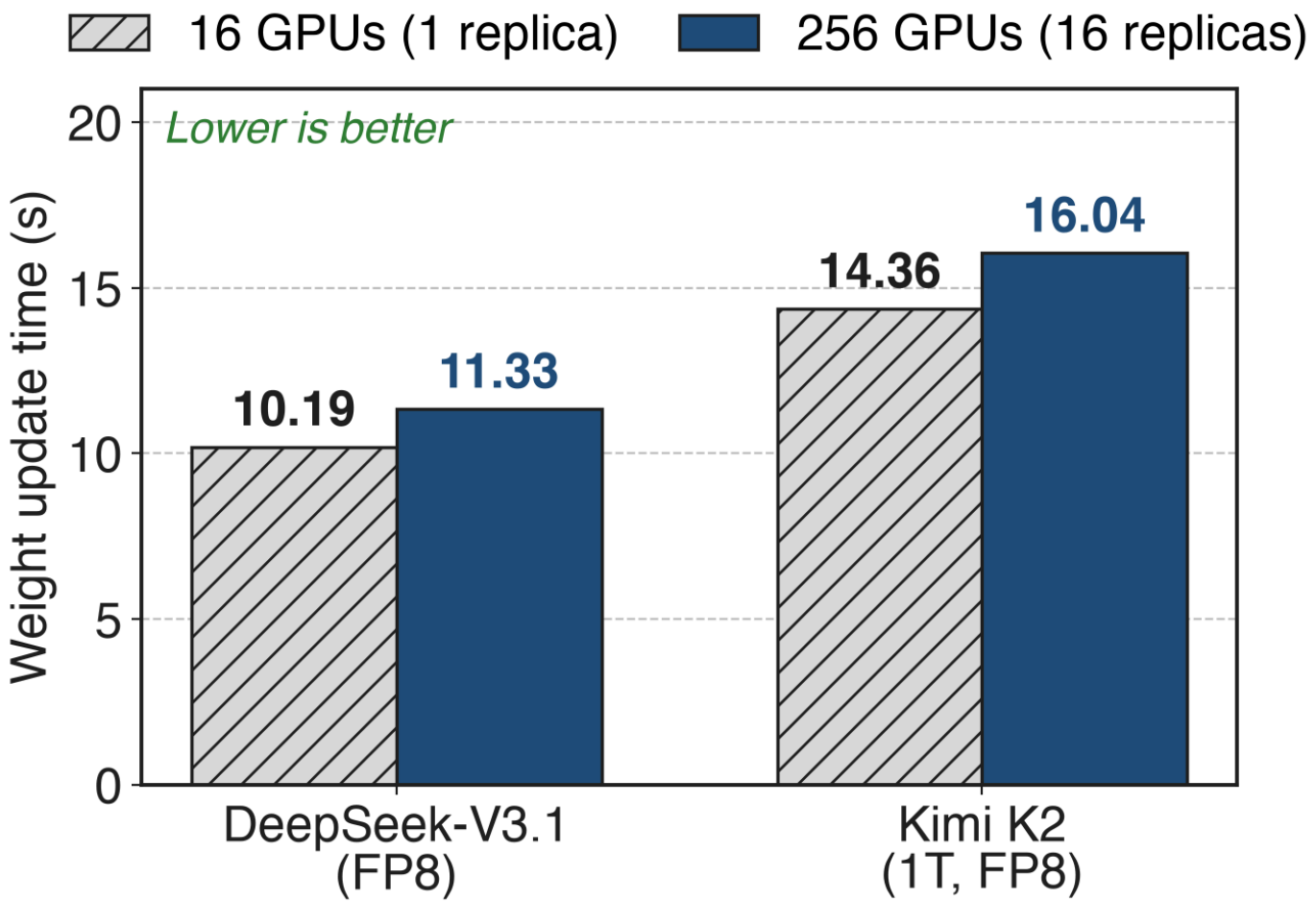
Optimized DCP cuts the save stall 12×

- Llama 3 405B, 432 H200 GPUs
- Cache plans and metadata; save from a process
- Local disks: full save ~126 s → ~47 s



K2 weights reshard into 16 replicas in 16 s

- Same idea at RL time: train layout → TP16 inference
- Broadcast once; each replica keeps its own shard
- 16× more GPUs, only 12% more time



The cost: collectives, cursors, 32-node quanta

- **DCP collectives**: dedup, metadata, reshard grow with scale
- **Data cursors** are per rank; new ranks resume weights only
- K2: elastic in 32-node steps; small runs offload optimizer
- No format guarantee across PyTorch versions

Weights and optimizer reshard freely; data position and RNG do not

Summary

- Even async saves stall training **18.5 s** per checkpoint (405B, 432 GPUs)
- **Insight**: shards are slices of global tensors, so any layout can read them
- **1.5 s** stall with optimized DCP; K2 runs on any multiple of **32** nodes
- Parent: deck 28 (VeOmni); related 54 (offload), 70 (FSDP)

https://github.com/ByteDance-Seed/VeOmni